Распределённый анализ вакансий DevOps в RustFS

Лабораторная работа 3

Выполнил: Смирнов Александр J3310

В работе построен воспроизводимый аналитический конвейер для варианта 11 «DevOps инженер»: исходный набор вакансий размещён в RustFS, частоты навыков, пар и троек рассчитаны собственной реализацией MapReduce и независимым SQL в DuckDB, после чего результаты сверены полностью. Обе реализации дали одинаковые 1 541 навык, 25 462 пары и 195 366 троек. Дополнительно проведены зарплатный и региональный анализ, три серии профилирования и контролируемый отказ одного маппера.

Исходные данные и индивидуальный вариант

Основной источник - it_vacancies_2024.csv объёмом 847 619 384 байта. В нём 229 388 уникальных вакансий без дубликатов по vacancy_id за период с 19 декабря 2023 года по 14 сентября 2024 года. Файл содержит 23 поля, включая регион, диапазон зарплаты, валюту, признак gross и ключевые навыки. Поэтому он подходит не только для частотного, но и для зарплатного и регионального этапов задания.

Выборка варианта формировалась по названию вакансии. В неё включены явные обозначения DevOps и DevSecOps, русское написание «девопс», а также функциональные синонимы SRE и Site Reliability Engineer. Platform Engineer включён только при точном совпадении словосочетания. Основную часть составили 5 653 названия с DevOps или DevSecOps, ещё 558 - SRE и Site Reliability, 4 - Platform Engineer. Такой критерий не использует описания вакансий и не подмешивает смежные роли только по отдельным навыкам.

```
regexp_matches(lower(coalesce(name, '')),
  'dev[[:space:]-]*(sec[[:space:]-]*)?ops|девопс|site reliability|
   (^|[^a-z])sre([^a-z]|$)|platform engineer')
```

Показатель	Значение
Все вакансии в CSV	229 388
Вакансии варианта 11	6 215
С непустым списком навыков	4 266
С сопоставимой зарплатой в RUR	1 364
Среднее число навыков на вакансию	5,66
Медиана и максимум навыков	5 и 30

Связь архива JSON с набором вакансий

Архив hh.zip содержит 10 000 JSON-файлов в полном формате API hh.ru. Проверка идентификаторов показала, что все 10 000 вакансий присутствуют в it_vacancies_2024.csv; следовательно, архив относится к тому же набору данных. Для демонстрации хранения отдельных объектов отобрана воспроизводимая случайная выборка из 1 000 JSON с seed=3310. Манифест фиксирует исходный список, seed, имена файлов и SHA-256.

Статистические расчёты выполнены по всем 6 215 вакансиям варианта, а не по тысяче JSON. Использование случайной тысячи для основной аналитики уменьшило бы выборку и сделало бы медианы и редкие сочетания зависимыми от случайного отбора.

JSON используются как дополнительное доказательство объектной загрузки, но не заменяют полный CSV.

Размещение данных в RustFS

Использован развёрнутый в предыдущей работе контейнер RustFS 1.0.0 с четырьмя папками-дисками. Создан приватный бакет hh-lake. На момент фиксации в нём находилось 1 020 объектов общим объёмом 871,9 MiB: полный CSV, каноническая выборка варианта, 1 000 JSON, результаты двух методов и журналы проверок. Ключи доступа в отчёт и репозиторий не включены; программы получают их из переменных окружения.

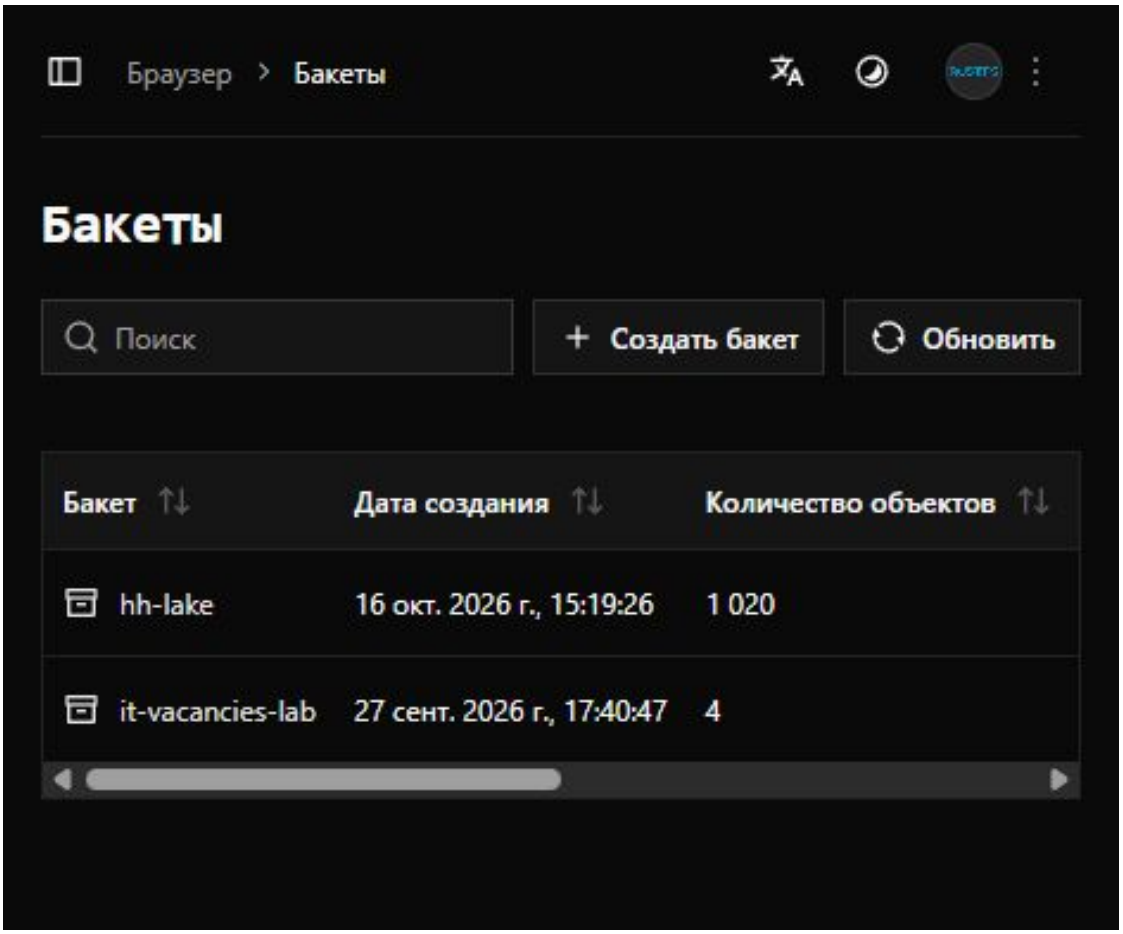


Рис. 1 - бакет hh-lake в веб-консоли RustFS

Префикс или объект	Назначение	Объём
raw/it_vacancies_2024.csv	полный исходный набор	847 619 384 байта
variants/devops/vacancies.ndjson	6 215 вакансий варианта	1 880 228 байт
supplementary/json-sample/	1 000 исходных JSON	5 435 694 байта
results/mapreduce/	навыки, пары и тройки	29 179 918 байт
results/duckdb/	контроль и аналитика	29 853 184 байта

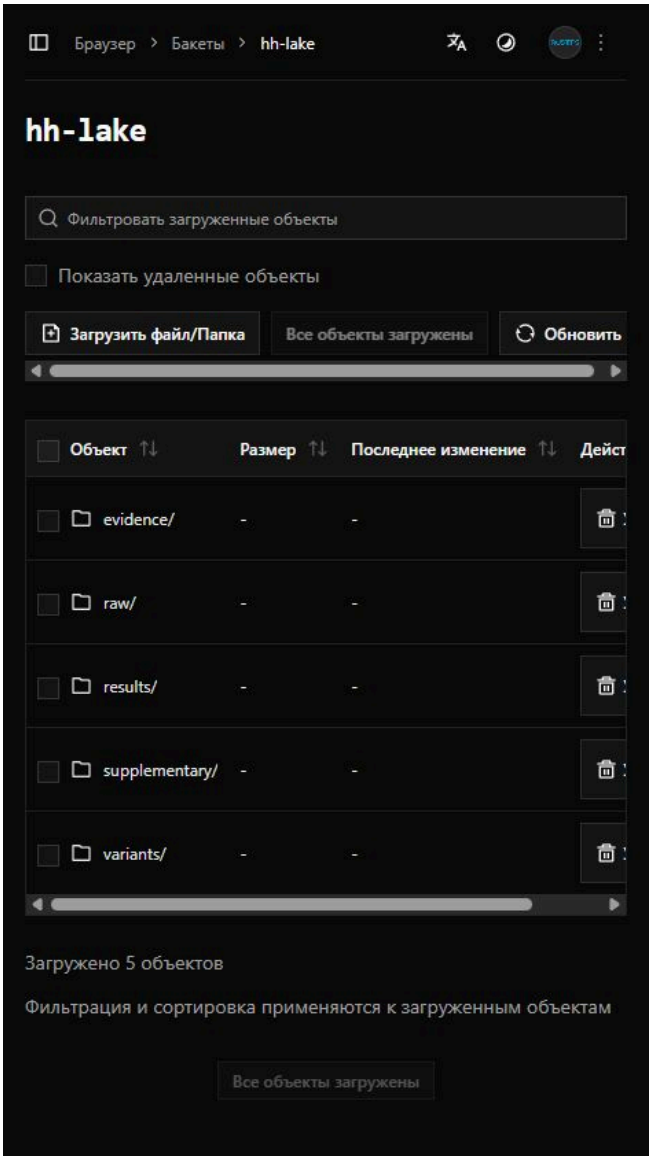


Рис. 2 - логическая структура данных в бакете hh-lake

На рис. 2 показаны пять S3-префиксов, существовавших до загрузки самого отчёта: evidence/, raw/, results/, supplementary/ и variants/. Строки являются префиксами, а не самостоятельными объектами, поэтому интерфейс обоснованно показывает дефисы вместо размера и времени изменения. После загрузки отчёта добавился префикс report/, а итоговое число объектов стало равно 1 020, как на рис. 1. Размеры фактических объектов приведены в таблице выше и подтверждены выводом ниже. Для исходного CSV веб-консоль показывает 808,4 MiB и время изменения 16 октября 2026 года, 15:19:36.

```
S3 inventory: 1020 objects, 914284187 bytes (871.9 MiB)
PUT s3://hh-lake/raw/it_vacancies_2024.csv
bytes: 847619384
etag: be9c1b4b3f57ff0e9771004c2a5fe78b-51
sha256: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
```

Подготовка данных

Навыки разделялись по точке с запятой, приводились к Unicode NFKC, очищались от повторных пробелов и сравнивались без учёта регистра. Внутри одной вакансии одинаковый нормализованный навык учитывался один раз. Исходное написание сохранялось в отдельном словаре меток, поэтому результаты остаются читаемыми.

Зарплаты сравнивались только для RUR. При наличии обеих границ использовалась середина диапазона, при одной границе - опубликованная граница. Значения net переводились в приблизительный gross делением на 0,87. Это допущение нужно для единой шкалы; оно не является расчётом фактического НДФЛ конкретного кандидата. Иностранные валюты и вакансии без числовой границы исключались.

Собственная реализация MapReduce

Файл NDJSON читается из RustFS четырьмя процессами с помощью S3 Range GET чанками по 524 288 байт. Граница чанка выравнивается по символу перевода строки, поэтому одна вакансия не теряется и не считается дважды. Mapper для вакансии с n уникальными навыками формирует n одиночных ключей, C(n,2) пар и C(n,3) троек. Reducer суммирует локальные счётчики процессов.

```
skills = json.loads(line)['skills']
groups = (
    skills,
    ('\u001f'.join(x) for x in combinations(skills, 2)),
    ('\u001f'.join(x) for x in combinations(skills, 3)),
)
for counter, values in zip(counters, groups):
    counter.update(values)
```

Базовый режим сначала передаёт все сгенерированные ключи родительскому процессу. Оптимизированный режим выполняет combiner внутри каждого mapper и передаёт только локальные словари частот. Это не меняет результат, но уменьшает объём межпроцессного обмена и память.

Независимый SQL в DuckDB

DuckDB подключается к RustFS через httpfs с path-style URL. Сначала самостоятельное SQL-условие варианта применяется к полному объекту s3://hh-lake/raw/it_vacancies_2024.csv. Оно выбрало 6 215 vacancy_id; расхождений с подготовленной Python-выборкой нет, а SHA-256 отсортированных наборов идентификаторов совпадает. Затем частоты считаются по тому же каноническому объекту варианта, что и MapReduce. Локальная копия для вычислений не используется. Одиночные навыки разворачиваются функцией unnest; пары и тройки строятся независимыми self join по vacancy_id с условиями строгого порядка.

```
SELECT DISTINCT vacancy_id
FROM read_csv_auto('s3://hh-lake/raw/it_vacancies_2024.csv',
                   header=true, all_varchar=true)
WHERE regexp_matches(lower(coalesce(name, '')),
  'dev[[:space:]-]*(sec[[:space:]-]*)?ops|девопс|site reliability|
   (^|[^a-z])sre([^a-z]|$)|platform engineer');
-- selected: 6215; sql_only: 0; prepared_only: 0

CREATE TABLE skill_rows AS
SELECT vacancy_id, unnest(skills) AS skill FROM vacancies;

SELECT a.skill AS skill1, b.skill AS skill2, count(*)
FROM skill_rows a JOIN skill_rows b USING (vacancy_id)
WHERE a.skill < b.skill
GROUP BY a.skill, b.skill;
```

Частоты навыков и сочетаний

Размер стека	Уникальных	Всего сгенерировано	Формула на вакансию
1 навык	1 541	35 158	n
2 навыка	25 462	173 204	C(n,2) = n(n-1)/2
3 навыка	195 366	687 102	C(n,3) = n(n-1)(n-2)/6

Число троек растёт значительно быстрее числа вакансий, потому что комбинации формируются внутри каждой вакансии. Например, при 5 навыках образуется 10 пар и 10 троек, а при 13 навыках - уже 78 пар и 286 троек. В выборке 90-й перцентиль равен 13 навыкам, максимум - 30. Поэтому 35 158 одиночных наблюдений превращаются в 173 204 наблюдения пар и 687 102 наблюдения троек.

Навык	Вакансий
Linux	2 745
Docker	2 336
Kubernetes	2 259
DevOps	1 577
Ansible	1 497
Python	1 276
CI/CD	1 249
PostgreSQL	1 157
Bash	1 135
Grafana	973



Рис. 3 - десять наиболее частых навыков

Наиболее частые пары и тройки

Пара	Вакансий
Docker + Linux	1 583
Docker + Kubernetes	1 549
Kubernetes + Linux	1 444
Ansible + Linux	1 069
DevOps + Linux	1 032
Ansible + Docker	990
DevOps + Docker	990
DevOps + Kubernetes	966

Тройка	Вакансий
Docker + Kubernetes + Linux	1 036
Ansible + Docker + Linux	717
DevOps + Docker + Kubernetes	699
DevOps + Docker + Linux	676
Ansible + Docker + Kubernetes	672
Ansible + Kubernetes + Linux	665
DevOps + Kubernetes + Linux	634
CI/CD + Docker + Kubernetes	543

Ядро сегмента образуют Linux, Docker и Kubernetes. Самая частая пара Docker + Linux встречается в 1 583 вакансиях, а тройка Docker + Kubernetes + Linux - в 1 036. Ansible, CI/CD, Python, PostgreSQL и средства наблюдаемости формируют второй слой требований. Частота навыка DevOps ниже частоты Linux, поскольку работодатели нередко указывают название роли, но не повторяют его в поле key_skills.

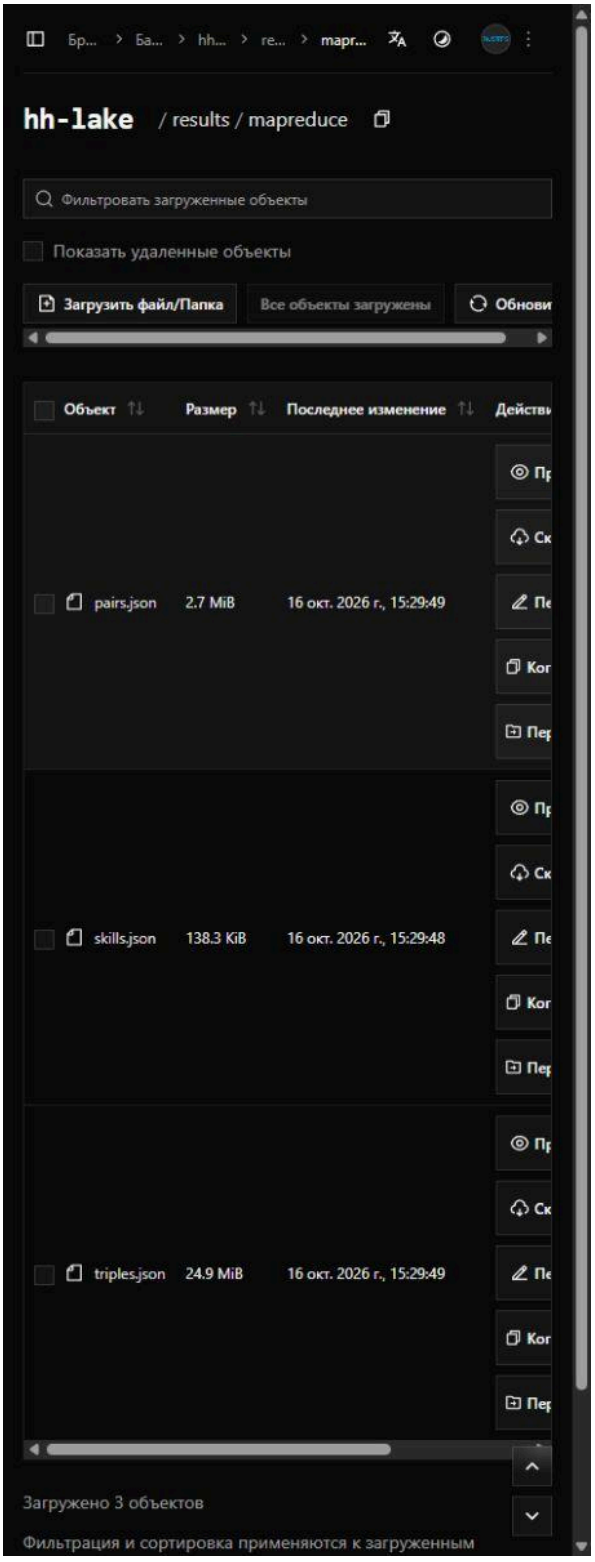


Рис. 4 - три результата MapReduce сохранены в hh-lake

Полная верификация результатов

Сравнение выполнено по полным JSON-документам после канонической сериализации, а не только по первым строкам или общим суммам. Для каждого размера стека совпали список ключей, порядок, каждая частота и SHA-256. Проверка завершилась all_equal=true.

Стек	Совпадение	SHA-256 канонического результата
Навыки	да	2ad809f9e9da9c73c8d311ace571b27b367b28cdf96b56d7e892e19615957e58
Пары	да	cafaec2c23af084d67ce40c8cf4e7847ab46bf54766cb1663d1213d500877a12
Тройки	да	4f194e88fc2c73d40541285a53d131d81b10b73f29fb257f9eaa01a9cd8bb4c2

```
all_equal: true
skills: 1541 unique, 35158 emissions
pairs: 25462 unique, 173204 emissions
triples: 195366 unique, 687102 emissions
```

Профилирование и оптимизация

Для каждого метода выполнено три независимых запуска на одном компьютере с четырьмя workers или threads. Во всех случаях обрабатывались один и тот же объект RustFS и одинаковая задача: подсчёт навыков, пар и троек с записью трёх JSON. Wall time измерялся вокруг дочернего процесса, а пик памяти - как максимальная сумма RSS дерева процессов при опросе каждые 10 мс. Для сравнения использована медиана трёх попыток.

Метод	Попытка	Wall time, с	Пик RSS, MiB
MapReduce базовый	1	2.278	417.2
	2	2.203	418.4
	3	2.196	420.4
MapReduce с combiner	1	2.107	362.3
	2	2.089	372.6
	3	2.129	365.4
DuckDB SQL	1	1.498	394.6
	2	1.478	394.3
	3	1.495	400.2

Метод	Медиана времени, с	Медиана пика, MiB
MapReduce базовый	2.203	418.4
MapReduce с combiner	2.107	365.4
DuckDB SQL	1.495	394.6

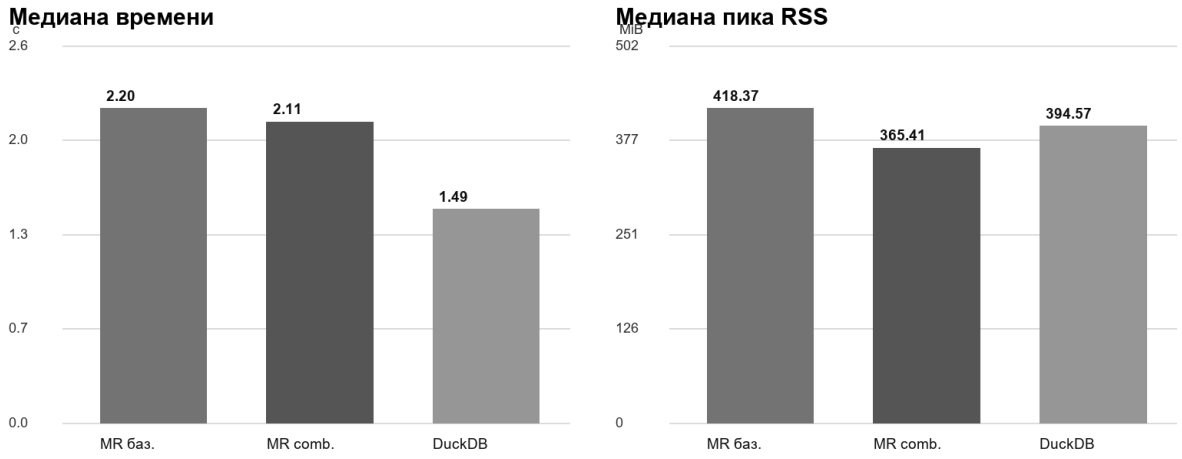


Рис. 5 - сравнение времени и пикового потребления памяти

Combiner ускорил собственную реализацию в 1,045 раза и снизил медианный пик памяти с 418,4 до 365,4 MiB, то есть на 12,66%. Выигрыш времени умеренный, потому что набор варианта невелик и заметную долю занимают запуск процессов, S3-запросы и сериализация результатов. Экономия памяти существеннее: повторяющиеся комбинации агрегируются до передачи reducer.

DuckDB выполнил ту же задачу за 1,495 с, что в 1,41 раза быстрее оптимизированного MapReduce, но потребовал 394,6 MiB - примерно на 8% больше. На этом объёме векторизованный SQL выигрывает по времени, а MapReduce с combiner - по пику памяти и явно демонстрирует распределение работы по чанкам. Результат относится к конкретному компьютеру и локальному RustFS; он не переносится автоматически на сетевой кластер.

Проверка отказа маппера

В chunk 1 на первой попытке программно вызвана ошибка. Координатор перезапустил только этот chunk; счётчик retries стал равен 1. После восстановления обработано ровно 6 215 вакансий, а SHA-256 всех трёх результатов совпал с обычным запуском. Тест проверяет retry и отсутствие двойного учёта.

```
chunks: 4
workers: 4
retries: 1
vacancies: 6215
result_after_retry_equal_to_normal_run: true
```

Влияние сочетаний навыков на зарплату

Общая медиана по 1 364 вакансиям с сопоставимой зарплатой составила 229 885 руб. gross в месяц. Для пары базой выбран более частый из двух отдельных навыков; для тройки - более частая из трёх входящих пар. Эффект равен разности медианы полного сочетания и медианы базового набора. В итоговые списки включены пары не менее чем с 20 зарплатными наблюдениями и тройки не менее чем с 15, чтобы не ранжировать единичные случаи.

Пары с наибольшим повышением и снижением

Пара	База	n	Медиана базы	Медиана пары	Разница
DevOps + Terraform	DevOps	62	229 885 руб.	344 828 руб.	+114 943 руб.
Terraform + Zabbix	Zabbix	35	229 885 руб.	344 828 руб.	+114 943 руб.
DevOps + ELK	DevOps	75	229 885 руб.	333 333 руб.	+103 448 руб.
Bash + ELK	Bash	39	229 885 руб.	316 092 руб.	+86 207 руб.
GitLab CI + PostgreSQL	PostgreSQL	31	229 885 руб.	316 092 руб.	+86 207 руб.
Debian + Nginx	Nginx	24	219 195 руб.	114 943 руб.	-104 253 руб.
Linux + Администрирование серверов	Linux	20	229 885 руб.	126 437 руб.	-103 448 руб.
Debian + Linux	Linux	32	229 885 руб.	128 966 руб.	-100 920 руб.
Linux + Администрирование сетевого оборудования	Linux	20	229 885 руб.	137 931 руб.	-91 954 руб.
PostgreSQL + Английский язык	PostgreSQL	20	229 885 руб.	150 000 руб.	-79 885 руб.

Наиболее высокий положительный сдвиг у DevOps + Terraform и Terraform + Zabbix: +114 943 руб. к медиане базового навыка. DevOps + ELK даёт +103 448 руб. Эти сочетания характерны для инфраструктуры как кода, production-наблюдаемости и платформ более высокой сложности. В нижней части находятся Debian + Nginx, Linux + администрирование серверов и Debian + Linux; они чаще описывают классические системно-администраторские задачи. Это ассоциация, а не доказательство причинного влияния навыка: на медиану также воздействуют уровень позиции, компания, регион и полнота публикации зарплаты.

Тройки с наибольшим повышением и снижением

Тройка	Базовая пара	n	Медиана базы	Медиана тройки	Разница
Bash + CI/CD + ELK	Bash + CI/CD	19	235 632 руб.	402 299 руб.	+166 667 руб.
Bash + DevOps + ELK	Bash + DevOps	27	244 253 руб.	402 299 руб.	+158 046 руб.
CI/CD + ELK + Gitlab	CI/CD + Gitlab	20	252 874 руб.	388 649 руб.	+135 776 руб.
Bash + ELK + Gitlab	Bash + Gitlab	21	273 000 руб.	402 299 руб.	+129 299 руб.
DevOps + GitLab CI + Nginx	DevOps + Nginx	17	220 000 руб.	344 828 руб.	+124 828 руб.
Bash + Kubernetes + MySQL	Bash + Kubernetes	21	287 356 руб.	172 414 руб.	-114 943 руб.
Bash + Nginx + SQL	Bash + Nginx	19	224 138 руб.	114 943 руб.	-109 195 руб.
Linux + SQL + Zabbix	Linux + Zabbix	15	221 264 руб.	114 943 руб.	-106 322 руб.
Bash + Debian + Linux	Bash + Linux	18	229 885 руб.	126 437 руб.	-103 448 руб.
Ansible + PostgreSQL + Английский язык	Ansible + PostgreSQL	15	247 126 руб.	143 678 руб.	-103 448 руб.

Наибольший положительный разрыв наблюдается для Bash + CI/CD + ELK: +166 667 руб. относительно более частой пары Bash + CI/CD. Bash + DevOps + ELK даёт +158

046 руб., CI/CD + ELK + GitLab - +135 776 руб. Отрицательные комбинации чаще включают MySQL, Debian, SQL без уточнения промышленной СУБД или административные формулировки. Минимальные пороги уменьшают случайность, но даже 15-20 наблюдений недостаточно для причинных выводов, поэтому результаты следует трактовать как направление для дальнейшей проверки.

Сравнение регионов

Для сравнения выбраны три региона с наибольшим числом вакансий варианта: Москва, Санкт-Петербург и Екатеринбург. Зарплатная медиана рассчитана только по сопоставимым RUR-значениям; востребованность навыка - как доля вакансий с непустым key_skills в соответствующем регионе.

```
SELECT area, count(*) AS vacancies,
       median(salary_mid_rur_gross) AS median_salary,
       count(*) FILTER (WHERE len(skills) > 0) AS with_skills,
       avg(len(skills)) AS avg_skills
FROM vacancies WHERE area IN ('Москва','Санкт-Петербург','Екатеринбург')
GROUP BY area ORDER BY vacancies DESC;
```

Регион	Вакансий	С зарплатой	Медиана gross	С навыками	Среднее навыков
Москва	3 536	699	287 356 руб.	2 355	5,25
Санкт-Петербург	841	217	229 885 руб.	593	5,96
Екатеринбург	231	43	183 908 руб.	164	6,14

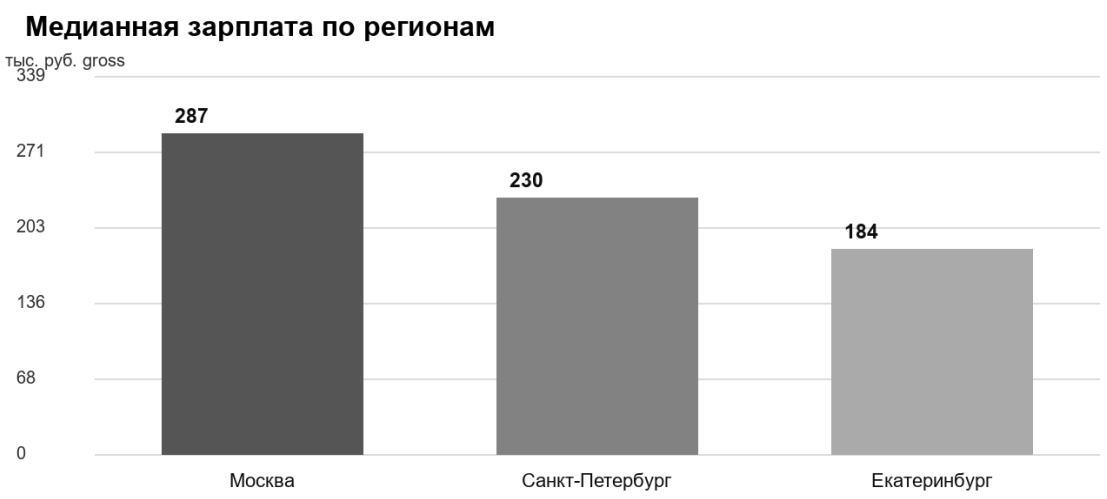


Рис. 6 - медианная зарплата в трёх регионах

Место	Москва	Санкт-Петербург	Екатеринбург
1	Linux - 61,57%	Linux - 65,26%	Linux - 67,68%
2	Kubernetes - 55,63%	Docker - 56,83%	Kubernetes - 57,32%
3	Docker - 52,53%	Kubernetes - 47,22%	Docker - 54,88%
4	DevOps - 37,15%	Ansible - 41,32%	DevOps - 37,2%
5	Ansible - 34,52%	DevOps - 32,88%	Ansible - 36,59%

Москва концентрирует 56,9% всех вакансий варианта и имеет медиану 287 356 руб. - на 25% выше Санкт-Петербурга и примерно на 56% выше Екатеринбурга. При этом

раскрытие зарплаты невысокое: 699 из 3 536 московских вакансий, поэтому медиана описывает только вакансии с опубликованной оплатой.

Linux занимает первое место во всех трёх регионах. В Москве Kubernetes встречается чаще Docker среди вакансий с навыками, в Санкт-Петербурге порядок обратный, а Екатеринбург близок к Москве по приоритету Kubernetes. Среднее число указанных навыков растёт от 5,25 в Москве до 6,14 в Екатеринбурге. Это может означать более широкие обязанности на меньшем рынке, но также отражать различия в стиле заполнения вакансий.

Ограничения анализа

- Зарплата опубликована только в 1 364 из 6 215 вакансий; пропуски могут быть систематическими.
- Одна граница зарплаты используется как оценка уровня, поэтому такие записи менее точны, чем полный диапазон.
- Поле key_skills заполняется работодателем свободно: встречаются синонимы, общие компетенции и опечатки. Нормализация регистра не объединяет разные термины автоматически.
- Разность медиан показывает ассоциацию, но не изолирует стаж, регион, компанию и другие смешивающие факторы.
- Профилирование относится к локальному стенду и не измеряет сетевое масштабирование нескольких узлов.

Выводы

Пайплайн выполнен полностью на данных варианта 11. Полный датасет и вспомогательная выборка JSON размещены в RustFS, а результаты трёх размеров стека сохранены в hh-lake. Собственный MapReduce действительно читает S3-объект по диапазонам и восстанавливается после отказа маппера. DuckDB независимо обращается к тому же S3-объекту и даёт побайтно эквивалентные логические результаты.

Рост числа сочетаний объясняется комбинаторикой длинных списков навыков. На локальном наборе DuckDB оказался быстрее, а MapReduce с combiner потребил меньше памяти, чем оба альтернативных режима. Зарплатный анализ связывает более высокие медианы с инфраструктурой как кодом, ELK и зрелыми CI/CD-стеками, а региональный анализ показывает заметную московскую премию при сходном ядре Linux, Docker и Kubernetes. Выводы ограничены неполным раскрытием зарплат и не интерпретируются как причинные.

Команды воспроизведения

```
python src/prepare_dataset.py it_vacancies_2024.csv --out-dir data/prepared
python src/s3_io.py upload data/prepared/devops_vacancies.ndjson variants/devops/vacancies.ndjson
python src/mapreduce_s3.py --out-dir results/mapreduce --mode optimized --workers 4 --chunk-bytes 524288
python src/duckdb_counts_s3.py --out-dir results/duckdb_counts_only --threads 4
python src/duckdb_analysis.py --out-dir results/duckdb --threads 4
python src/duckdb_variant_check.py --out evidence/duckdb_variant_check.json
python src/verify_results.py results/mapreduce results/duckdb_counts_only --out evidence/verification.json
python src/profile_pipeline.py --attempts 3 --out evidence/profiling.json
```

Источники

RustFS docs: https://docs.rustfs.com/
DuckDB S3 API: https://duckdb.org/docs/stable/core_extensions/httpfs/s3api
DuckDB JSON: https://duckdb.org/docs/stable/data/json/loading_json
Python itertools.combinations:
https://docs.python.org/3/library/itertools.html#itertools.combinations